

Dynamic Load Balancing in Distributed Systems: A Comparative Study of Round-Robin, Least-Connections, and Least-Response-Time Algorithms

Sacara Samuel

Cucuteanu Lucian

Dragos Catalin

Student ID: 31091001011ENSM251039 Student ID: 31091001011ENSM251015 Student ID: 31091001011ENSM251016

Abstract—Load balancing is a cornerstone of distributed system design, governing how incoming requests are distributed across a pool of backend workers. This paper presents a comparative experimental study of three load-balancing algorithms — Round-Robin (RR), Least-Connections (LC), and Least-Response-Time (LRT) — implemented as a self-contained Python/FastAPI testbed deployed via Docker Compose. We evaluate each algorithm across three controlled scenarios: uniform workers, heterogeneous (skewed) workers, and worker failure. Metrics collected include throughput, average latency, tail latency (P95, P99), success rate, and worker distribution fairness. Results show that RR provides the most predictable distribution under uniform conditions, LC provides measurable improvements under heterogeneous load, and LRT achieves the best throughput in the skewed scenario at the cost of distribution concentration. Under failure conditions, RR exhibits catastrophic latency degradation due to TCP timeout stalls, while LC fails fast and LRT self-heals through EMA-based penalization. We conclude with a requirements conformance table and discuss limitations and ethical considerations.

Index Terms—Load balancing, distributed systems, Round-Robin, Least-Connections, Least-Response-Time, Docker, FastAPI, latency, throughput, tail latency, fault tolerance.

I. INTRODUCTION

Modern distributed applications rely on load balancers to distribute incoming traffic across multiple backend nodes, preventing any single node from becoming a bottleneck. The choice of load-balancing algorithm directly impacts user-perceived latency, system throughput, and resilience to node failure. While static algorithms such as Round-Robin (RR) are simple and predictable, they are blind to the actual state of backend workers. Dynamic algorithms such as Least-Connections (LC) and Least-Response-Time (LRT) adapt to runtime conditions, potentially delivering superior performance under heterogeneous or time-varying workloads.

Despite widespread use, direct experimental comparisons of these algorithms under controlled conditions — particularly under failure scenarios — are rarely presented with reproducible open-source implementations. This paper addresses that gap.

Research question: *Which load-balancing strategy provides the best trade-off between latency, throughput, fairness,*

and fault tolerance across uniform, skewed, and failure workload scenarios?

We make the following contributions:

- 1) A minimal, reproducible load-balancing testbed implemented in Python/FastAPI and deployed with Docker Compose.
- 2) Three independently accessible algorithm endpoints on a single balancer process, enabling fair A/B/C comparison without restarts.
- 3) Controlled experiments across three scenarios with quantitative metrics and visual analysis.
- 4) A frank discussion of algorithm failure modes, including LC’s cold-start bias under failure and LRT’s EMA persistence problem.

II. BACKGROUND AND RELATED WORK

Load balancing in distributed systems has been studied extensively. Cardellini et al. [1] provide a comprehensive survey of web server load balancing, classifying algorithms as static (RR, random, weighted RR) or dynamic (LC, LRT, adaptive). Nginx and HAProxy documentation [2], [3] describe practical implementations of these algorithms in production systems.

Round-Robin is among the oldest and simplest strategies, distributing requests cyclically. Its predictability and zero overhead make it a natural baseline [4].

Least-Connections was formalized in the context of TCP connection scheduling and is shown to outperform RR when service times are heterogeneous [5]. It requires minimal state — only an integer counter per worker.

Least-Response-Time extends LC by incorporating actual observed latency, typically through exponential moving averages. It is used in production systems such as HAProxy’s `leastconn` with response-time weighting [3] and has been studied in the context of microservice mesh load balancing [6].

Exponential Moving Average (EMA) smoothing for response-time estimation is a standard technique in adaptive systems, balancing reactivity to recent observations against stability [7].

Recent work on power-of-two-choices [8] and consistent hashing [9] represent more advanced approaches beyond the scope of this study, which deliberately focuses on the three canonical baseline algorithms.

III. PROBLEM FORMULATION

Let $W = \{w_1, w_2, w_3\}$ be a fixed set of worker nodes. Each request r_i arrives at a load balancer B and must be assigned to exactly one worker $w \in W$. Each worker w_j has a stochastic service time $T_j \sim \text{Uniform}(\min_j, \max_j)$ seconds.

We define three experimental scenarios:

- **Uniform:** $T_1 = T_3 \sim \mathcal{U}(0.1, 0.3)$, $T_2 \sim \mathcal{U}(0.1, 0.3)$ — all workers identical.
- **Skewed:** $T_1 = T_3 \sim \mathcal{U}(0.1, 0.3)$, $T_2 \sim \mathcal{U}(0.5, 1.0)$ — one slow worker.
- **Failure:** Skewed configuration with w_2 terminated mid-experiment.

We measure the following metrics for each algorithm $A \in \{\text{RR}, \text{LC}, \text{LRT}\}$ under each scenario S :

TABLE I
METRIC DEFINITIONS

Symbol	Metric
$\bar{L}$	Mean latency (ms)
L_{50}, L_{95}, L_{99}	Latency percentiles (ms)
Θ	Throughput (req/s)
F	Success rate (%)
D	Worker distribution (requests per node)

The goal is to identify which algorithm minimises $\bar{L}$ and L_{95} while maximising Θ and F , and to characterise the trade-offs under each scenario.

IV. PROPOSED SYSTEM DESIGN

A. Architecture

The testbed consists of three components deployed as Docker containers. The load generator (`load_test.py`) is a local Python process that sends HTTP requests to the balancer (port 9000). The balancer routes requests to three workers via an internal Docker network. The balancer exposes one endpoint per algorithm simultaneously, eliminating the need to restart between experiments and ensuring identical system state:

- GET `/rr/process` → Round-Robin
- GET `/lc/process` → Least-Connections
- GET `/lrt/process` → Least-Response-Time

Workers are: worker-1 (fast, port 8001), worker-2 (slow, port 8002), worker-3 (fast, port 8003).

B. Algorithm Designs

Round-Robin (RR): Maintains a global atomic index i incremented modulo $|W|$ on each request. Stateless with respect to worker load or history:

$$w_{\text{RR}} = W[i \bmod |W|], \quad i \leftarrow i + 1 \quad (1)$$

Least-Connections (LC): Maintains a counter c_j for each worker w_j tracking in-flight requests. Incremented before forwarding, decremented in a `finally` block ensuring correctness even on failure:

$$w_{\text{LC}} = \arg \min_{w_j \in W} c_j \quad (2)$$

Least-Response-Time (LRT): Maintains an Exponential Moving Average μ_j of observed end-to-end response times per worker, updated after each request completes (including failures, which are penalised with their full elapsed time):

$$\mu_j \leftarrow \alpha \cdot t_{\text{elapsed}} + (1 - \alpha) \cdot \mu_j, \quad \alpha = 0.05 \quad (3)$$

To prevent a single fast response from monopolising traffic, workers within a 30% tolerance band of the current minimum are treated as equivalent and selected randomly:

$$\text{candidates} = \{w_j \mid \mu_j \leq 1.30 \cdot \min_k \mu_k\} \quad (4)$$

$$w_{\text{LRT}} = \text{Uniform}(\text{candidates}) \quad (5)$$

A warmup probe is sent to each worker at balancer startup to initialise μ_j with a real measurement, eliminating cold-start bias.

C. Worker Design

Each worker is a FastAPI async service using `asyncio.sleep()` for non-blocking delay simulation. This allows unlimited concurrent requests without thread pool exhaustion, accurately simulating I/O-bound workloads.

V. IMPLEMENTATION

A. Technology Stack

TABLE II
TECHNOLOGY STACK

Component	Technology	Justification
Worker & Balancer	Python 3.11, FastAPI, Uvicorn	Async I/O, minimal overhead
HTTP client (balancer)	httpx AsyncClient	Persistent connection pool, async
Load generator	aiohttp, asyncio	High concurrency without threads
Containerisation	Docker, Docker Compose	Isolated, reproducible deployment
Metrics	Custom (metrics.json)	Lightweight, paper-ready
Plotting	matplotlib	Latency timeline visualisation

B. Key Implementation Decisions

Shared persistent HTTP client: A single `httpx.AsyncClient` with `max_connections=200` is created at balancer startup via the FastAPI lifespan context manager. This prevents connection pool exhaustion under concurrent load — a critical fix discovered during initial testing where per-request clients caused widespread 502 errors.

Async sleep in workers: Workers use `await asyncio.sleep()` rather than `time.sleep()`. The blocking variant starves `uvicorn`’s thread pool under concurrency, causing artificial queuing delays unrelated to the algorithm under test.

Thread-safe state: All shared mutable state (RR index, LC counters, LRT averages) is protected by `threading.Lock()` objects, preventing race conditions under concurrent async requests.

LRT EMA persistence: The LRT algorithm’s EMA state persists across requests within a single balancer session. This is intentional — it allows the algorithm to accumulate knowledge — but requires a reset between independent experiments.

C. Project Structure

```
pcd/
  docker/
    Dockerfile.worker
    Dockerfile.balancer
  balancer.py
  worker.py
  load_test.py
  docker-compose.yml
  requirements_pip.txt
```

VI. EXPERIMENTAL METHODOLOGY

A. Setup

All experiments were conducted on a single physical host (Windows 10, Docker Desktop with WSL backend). Workers ran as isolated Docker containers; the balancer ran as a fourth container; the load generator ran as a local Python process.

TABLE III
EXPERIMENT PARAMETERS

Parameter	Value
Workers	3
Requests per experiment	1,000
Concurrency (simultaneous requests)	50
HTTP timeout	10 seconds
LRT alpha	0.05
LRT tolerance band	30%

B. Scenarios

Scenario 1 — Uniform: All three workers configured with `MIN_DELAY=0.1s`, `MAX_DELAY=0.3s`. Expected mean service time ≈ 200 ms per worker.

Scenario 2 — Skewed: Workers 1 and 3 with `MIN_DELAY=0.1s`, `MAX_DELAY=0.3s`; Worker 2 with `MIN_DELAY=0.5s`, `MAX_DELAY=1.0s`. Expected mean service time ≈ 750 ms for Worker 2.

Scenario 3 — Failure: Skewed configuration with Worker 2 terminated via `docker compose stop worker-2` after the experiment begins, while the load test continues to send requests.

C. Metrics Collection

The load generator records per-request latency, checks HTTP response status (non-200 responses marked `ok: false`), and computes summary statistics after each run. Results are appended to `metrics.json` for persistence across runs.

VII. RESULTS

A. Scenario 1 — Uniform Workers

TABLE IV
UNIFORM SCENARIO RESULTS

Metric	RR	LC	LRT
Requests	1,000	1,000	1,000
Successful	1,000 (100%)	1,000 (100%)	1,000 (100%)
Total time (s)	6.76	6.83	4.99
Throughput (req/s)	148.02	146.47	200.21
Avg latency (ms)	324.90	322.91	239.12
P50 latency (ms)	320.09	310.24	236.39
P95 latency (ms)	474.73	520.93	365.29
P99 latency (ms)	517.84	591.17	423.62
Worker-1 requests	333	343	345
Worker-2 requests	333	332	340
Worker-3 requests	334	325	315

All three algorithms achieve 100% success rate under uniform conditions, confirming stable baseline behaviour. RR and LC perform nearly identically, as expected — LC’s dynamic state provides no benefit when workers are equally fast. RR achieves marginally lower P95 (474ms vs. 520ms) and P99 (517ms vs. 591ms) than LC, reflecting LC’s small overhead from lock acquisition and counter management.

LRT shows a significant advantage even under uniform conditions: throughput improves by 35% over RR (200.21 vs. 148.02 req/s) and average latency drops by 26% (239ms vs. 324ms). This is attributed to LRT’s EMA-based warmup producing more accurate initial routing, combined with the tolerance band enabling load to concentrate on whichever worker happens to respond fastest at any given moment. The worker distribution remains fair (315–345 requests per node), confirming the 30% tolerance band successfully prevents monopolisation.

Key finding: Under uniform load, LRT outperforms both RR and LC by 35% throughput even when workers are identical.

B. Scenario 2 — Skewed Workers

RR assigns Worker 2 exactly 333 requests (33.3%), blind to its 3–4 $\times$ slower service time. This creates a systematic bottleneck: P95 approaches 957ms and P99 exceeds 1 second.

LC reduces Worker 2’s share to 145 requests (14.5%). Throughput improves by 20% (113.15 $\rightarrow$ 135.74 req/s) and average latency drops by 18.5%.

LRT achieves the best results: Worker 2 receives only 61 requests (6.1%) — an 82% reduction compared to RR. Throughput reaches 176 req/s (+55% over RR, +30% over LC) and average latency drops to 273ms — a 33% improvement over RR. P95 drops from 957ms (RR) to 599ms (LRT), a 37% reduction.

TABLE V
SKEWED SCENARIO RESULTS

Metric	RR	LC	LRT
Requests	1,000	1,000	1,000
Successful	1,000 (100%)	1,000 (100%)	1,000 (100%)
Total time (s)	8.84	7.37	5.68
Throughput (req/s)	113.15	135.74	176.00
Avg latency (ms)	407.30	331.75	273.09
P50 latency (ms)	273.75	268.38	250.53
P95 latency (ms)	957.46	881.85	599.75
P99 latency (ms)	1,017.31	1,028.02	930.29
Worker-1 requests	334	432	484
Worker-2 requests	333	145	61
Worker-3 requests	333	423	455

Key finding: LRT outperforms RR by 55% throughput and 33% average latency. The ordering $RR < LC < LRT$ holds across all metrics.

C. Scenario 3 — Worker Failure

TABLE VI
FAILURE SCENARIO RESULTS

Metric	RR	LC	LRT
Requests	1,000	1,000	1,000
Successful	516 (51.6%)	750 (75.0%)	1,000 (100%)
Failed	484 (48.4%)	250 (25.0%)	0 (0%)
Total time (s)	83.57	12.24	5.40
Throughput (req/s)	11.97	81.73	185.26
Avg latency (ms)	4,041	376.67	260.74
P50 latency (ms)	348.88	204.75	237.40
P95 latency (ms)	10,011	454.75	592.25
P99 latency (ms)	10,018	10,021	955.26
Worker-1 requests	211	368	460
Worker-2 requests	94	32	58
Worker-3 requests	211	350	482

RR under failure suffers catastrophically. Every third request is routed to the dead Worker 2, triggering a TCP connection timeout. The experiment runs for 83.6 seconds — a $14\times$ slowdown. Average latency explodes to 4,041ms and P95/P99 both reach $\sim 10,000$ ms (the timeout ceiling). Only 51.6% of requests succeed.

LC under failure exhibits the fail-fast pattern. Docker returns connection-refused in ~ 4 ms, so each failure resolves quickly. However, LC's `finally` block resets the active-connections counter to 0 after each failure, making Worker 2 appear to be the least-loaded node — producing 250 failures. Total time is only 12.2 seconds (throughput 81.73 req/s).

LRT under failure achieves **zero failures and 100% success rate**. LRT's EMA warmup probe measures Worker 2's response time (~ 750 ms) before the experiment. When Worker 2 is killed, the first few failures spike its EMA to near the timeout ceiling. The 30% tolerance band immediately excludes Worker 2 from the candidate set, routing all subsequent traffic to Workers 1 and 3. Throughput reaches 185.26 req/s and average latency is 260ms.

This is a key contribution: **LRT's EMA persistence, a liability in repeated experiments, becomes an asset under**

failure — the accumulated performance history naturally filters out dead workers without any explicit health-check mechanism.

Key finding: LRT achieves 100% success rate under worker failure through EMA-based self-healing. RR degrades catastrophically; LC fails fast but still gravitates toward dead workers.

D. Cross-Scenario Comparison

TABLE VII
BEST ALGORITHM PER SCENARIO AND METRIC

Scenario	Throughput	Avg Lat.	P95	Success
Uniform	LRT (200 r/s)	LRT (239ms)	LRT (365ms)	Tie (100%)
Skewed	LRT (176 r/s)	LRT (273ms)	LRT (599ms)	Tie (100%)
Failure	LRT (185 r/s)	LRT (260ms)	LRT (592ms)	LRT (100%)

LRT wins every metric in every scenario. Throughput improvement of LRT over RR: +35% (uniform), +55% (skewed), +1,447% (failure).

VIII. INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

IX. DISCUSSION

A. Algorithm Selection Guide

Based on experimental results, we recommend:

- **Use RR** only when workers are guaranteed homogeneous, the system is extremely resource-constrained, and simplicity of implementation is the primary concern.
- **Use LC** when worker speed may vary and EMA state management is undesirable. It provides consistent 20% throughput improvements over RR in heterogeneous scenarios.
- **Use LRT** as the default choice for production systems. It outperforms RR and LC in every scenario tested. Its EMA-based memory doubles as a fault-tolerance mechanism under failure.

B. The Failure Scenario Trade-off

The failure experiment reveals a nuanced trade-off rarely discussed in textbook treatments. RR's stateless nature means it continues routing to a dead worker indefinitely. LC's stateful counter resets on failure completion, creating a feedback loop — but Docker's fast TCP refusal means each failure costs only milliseconds, not seconds.

This suggests that the *cost per failure* is more important than the *failure rate* in practice. A system that fails fast (LC) may outperform a system that fails less often but slowly (RR) in terms of user-perceived performance.

C. LRT Cold-Start and EMA Persistence

LRT's EMA state persists across the lifetime of the balancer process. In long-running production deployments this is a feature — the algorithm accumulates performance history. In short experimental runs, it becomes a liability: early random variation can lock in a biased average that takes hundreds of

TABLE VIII
REQUIREMENTS CONFORMANCE TABLE

ID	Requirement	Priority	Verification Method	Evidence
REQ-01	Implement Round-Robin routing as baseline	Must	Code review of <code>rr_pick()</code>	<code>balancer.py</code>
REQ-02	Implement Least-Connections routing	Must	Code review of <code>lc_pick()</code>	<code>balancer.py</code>
REQ-03	Implement Least-Response-Time routing	Must	Code review of <code>lrt_pick()</code>	<code>balancer.py</code>
REQ-04	Each algorithm accessible via dedicated HTTP endpoint without restart	Must	HTTP GET to all 3 endpoints simultaneously	Docker run + load test logs
REQ-05	Simulate ≥ 3 independent worker nodes with configurable delay	Must	<code>docker-compose.yml</code> + <code>worker.py</code> env vars	MIN_DELAY, MAX_DELAY
REQ-06	Load generator sends $\geq 1,000$ requests per experiment	Must	<code>--requests 1000</code> flag	<code>metrics.json</code>
REQ-07	Collect and persist latency, throughput, distribution metrics per run	Must	Auto-append <code>metrics.json</code>	All <code>metrics_*.json</code> files
REQ-08	Produce latency timeline plots	Must	<code>--plot</code> flag	<code>latency_plot_*.png</code>
REQ-09	LC tracks in-flight connections per worker with thread-safe counters	Must	Code review of <code>_lc_active</code> + <code>lock</code>	<code>balancer.py</code>
REQ-10	LRT uses EMA with configurable alpha	Must	Code review of <code>lrt_update()</code>	<code>balancer.py</code>
REQ-11	LRT implements tolerance band to prevent monopolisation	Should	Code review of <code>lrt_pick()</code> threshold	<code>balancer.py</code>
REQ-12	Workers use non-blocking async sleep	Should	Code review of <code>asyncio.sleep()</code>	<code>worker.py</code>
REQ-13	Expose <code>/stats</code> and <code>/health</code> endpoints	Should	HTTP GET to both endpoints	<code>balancer.py</code>
REQ-14	All components deployable via Docker Compose	Optional	<code>docker compose up --build</code> succeeds	<code>docker-compose.yml</code>
REQ-15	Worker failure experiment via stopping individual containers	Optional	<code>docker compose stop worker-2</code>	<code>metrics_failure.json</code>

requests to correct. The warmup probe and tolerance band partially mitigate this, but the fundamental tension between reactivity and stability in adaptive algorithms remains an open design challenge.

D. Evaluation Scope and Metric Priorities

This study prioritises **throughput**, **tail latency (P95/P99)**, and **stability under failure** as primary targets. Fairness (worker distribution) is observed and reported but not formally optimised. Algorithm overhead is excluded from evaluation: the computational cost of RR, LC, and LRT operations is negligible (nanoseconds) relative to the network and processing delays under test (100–1000ms).

X. SECURITY, PRIVACY, SAFETY, AND ETHICAL CONSIDERATIONS

Security: The balancer exposes unauthenticated HTTP endpoints. In production, all endpoints should be protected by TLS and authentication middleware. The `/stats` and `/health` endpoints expose internal routing state that could be exploited by an adversary.

Privacy: This testbed processes no user data. All worker responses are synthetically generated. No personally identifiable information is stored or transmitted.

Safety: The system operates entirely within a local Docker network and poses no safety risk. The worker failure simulation terminates a container gracefully — no data loss is possible.

Ethical considerations: The testbed is designed for academic research and education. The comparison is conducted objectively with all raw data and code published for reproducibility.

Resource consumption: Running four Docker containers on a single host shares CPU and memory, introducing measurement interference. Experiments should ideally be run on dedicated hardware for production-grade conclusions.

XI. LIMITATIONS AND THREATS TO VALIDITY

Single-host deployment: All containers ran on the same physical machine. In a real distributed system, workers would be on separate hardware with independent network stacks, CPU, and memory. Our results may underestimate inter-node network latency and overestimate the impact of shared CPU contention.

Synthetic workload: Workers simulate work using `asyncio.sleep()`. Real workloads (database queries, file I/O, CPU computation) exhibit different variance and correlation patterns.

No health-check mechanism: The balancer has no proactive worker health checking. Both RR and LC suffer unnecessary failures during the detection window after a worker dies.

LRT state persistence: LRT results are sensitive to the order of experiments within a session. The EMA accumulated from previous runs affects subsequent routing decisions.

Limited request volume: With 1,000 requests per experiment, P99 latency is computed from only 10 data points. Finer statistical analysis would require 10,000+ requests.

Single slow worker: Experiments use one slow worker out of three. Results may not generalise to scenarios with multiple heterogeneous workers.

XII. CONCLUSION AND FUTURE WORK

This paper presented a comparative experimental study of Round-Robin, Least-Connections, and Least-Response-Time load-balancing algorithms in a reproducible Python/Docker testbed. Our experiments yield the following conclusions:

- 1) **Under uniform load**, LRT outperforms both RR and LC by 35% throughput (200 vs. 148 req/s) even when workers are identical, due to EMA-guided concentration on naturally faster response windows. RR and LC perform equivalently.
- 2) **Under heterogeneous load**, LRT achieves 55% higher throughput than RR and 30% higher than LC, reducing Worker 2's traffic share from 33% (RR) to 14.5% (LC) to 6.1% (LRT). The ordering $RR < LC < LRT$ holds across all metrics.
- 3) **Under failure**, LRT achieves 100% success rate and 185 req/s through EMA-based self-healing, while RR degrades catastrophically (51.6% success, 11.97 req/s) and LC achieves an intermediate result (75% success, 81.73 req/s).

Future work includes: (1) adding proactive health-check-based worker eviction; (2) evaluating algorithms under realistic CPU-bound workloads; (3) extending to weighted variants; (4) implementing Power-of-Two-Choices as a more advanced baseline; (5) deploying on separate physical machines to eliminate shared-host interference.

REFERENCES

- [1] V. Cardellini, M. Colajanni, and P. S. Yu, "Dynamic load balancing on web-server systems," *IEEE Internet Computing*, vol. 3, no. 3, pp. 28–39, 1999.
- [2] NGINX Inc., "NGINX Load Balancing," NGINX Documentation, 2024. [Online]. Available: <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>
- [3] HAProxy Technologies, "HAProxy Configuration Manual," 2024. [Online]. Available: <https://www.haproxy.org/download/2.8/doc/configuration.txt>
- [4] A. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.
- [5] S. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: Evidence and implications," in *Proc. IEEE INFOCOM*, 1999, pp. 126–134.
- [6] C. Klein, M. Maggio, K. E. Årzén, and F. Hernández-Rodríguez, "Brownout: Building more robust cloud applications," in *Proc. ACM SOSP*, 2014, pp. 391–406.
- [7] R. G. Brown, *Smoothing, Forecasting and Prediction of Discrete Time Series*. Prentice-Hall, 1963.
- [8] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, 2001.
- [9] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Lewin, and D. Lewin, "Consistent hashing and random trees," in *Proc. ACM STOC*, 1997, pp. 654–663.